Pygame Space Invaders lab

Full chat export - Friday, 23 October 2026

User

I'm working on a Pygame Space Invaders clone for a college lab. I'll give you the full code of every file. Please read it all and don't change anything yet. Summarise in a few lines how the game loop, bullets, enemies and collision detection currently work, and list any bugs or weak spots you notice. Then wait for my next instruction.

Rules for all your later answers: give me the full updated code for every file you change (not snippets), tell me which file each block belongs to, keep the existing folder structure and class names, and keep the code simple and well commented.

--- main.py ---

```
import pygame
from game.game_engine import GameEngine

# Initialize pygame/Start application
pygame.init()

# Screen dimensions
WIDTH, HEIGHT = 600, 700
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Space Invaders - Pygame Version")

# Colors
BLACK = (0, 0, 0)

# Clock
clock = pygame.time.Clock()
FPS = 60

# Game loop
engine = GameEngine(WIDTH, HEIGHT)

def main():
    running = True
    while running:
        SCREEN.fill(BLACK)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            engine.handle_event(event)

        engine.handle_input()
        engine.update()
        engine.render(SCREEN)

        pygame.display.flip()
        clock.tick(FPS)

    pygame.quit()

if __name__ == "__main__":
    main()
```

--- game/game_engine.py ---

```
import pygame
import random
from .player import Player
from .enemy import EnemyGrid
from .bullet import Bullet

# Game Engine

WHITE = (255, 255, 255)
GREEN = (0, 200, 0)
```

```
RED = (220, 60, 60)

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.player = Player(width // 2 - 20, height - 50, 40, 20)
        self.enemy_grid = EnemyGrid(width)

        self.player_bullets = []
        self.enemy_bullets = []
        self._shoot_cooldown = 0
        self.enemy_fire_chance = 0.01

        self.score = 0
        self.font = pygame.font.SysFont("Arial", 30)
        self.game_over = False

    def handle_event(self, event):
        if event.type == pygame.KEYDOWN and event.key == pygame.K_SPACE:
            if self._shoot_cooldown <= 0:
                bullet_x = self.player.center_x() - 2
                self.player_bullets.append(Bullet(bullet_x, self.player.y, direction=-1))
                self._shoot_cooldown = 15

    def handle_input(self):
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT] or keys[pygame.K_a]:
            self.player.move(-self.player.speed, self.width)
        if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
            self.player.move(self.player.speed, self.width)

    def update(self):
        if self.game_over:
            return

        if self._shoot_cooldown > 0:
            self._shoot_cooldown -= 1

        self.enemy_grid.move()

        for enemy in self.enemy_grid.alive_enemies():
            if random.random() < self.enemy_fire_chance:
                bullet_x = enemy.x + enemy.width // 2
                self.enemy_bullets.append(Bullet(bullet_x, enemy.y + enemy.height, direction=1))

        for bullet in self.player_bullets:
            bullet.move()
        for bullet in self.enemy_bullets:
            bullet.move()

        self.player_bullets = [b for b in self.player_bullets if not b.off_screen(self.height)]
        self.enemy_bullets = [b for b in self.enemy_bullets if not b.off_screen(self.height)]

        # NOTE: this removes a bullet from player_bullets while iterating
        # directly over that same list. Python skips the element right
        # after a removed one, so when two enemies are hit on the same
        # frame the second collision can be missed - the bullet appears
        # to pass straight through. See Task 1 in the README.
        for bullet in self.player_bullets:
            for enemy in self.enemy_grid.alive_enemies():
                if bullet.rect().colliderect(enemy.rect()):
                    enemy.alive = False
                    self.player_bullets.remove(bullet)
                    self.score += 1
                    break

        for bullet in self.enemy_bullets:
            if bullet.rect().colliderect(self.player.rect()):
                self.game_over = True
                break

        if self.enemy_grid.reached_bottom(self.player.y):
```

```
            self.game_over = True

    def render(self, screen):
        pygame.draw.rect(screen, GREEN, self.player.rect())

        for enemy in self.enemy_grid.alive_enemies():
            pygame.draw.rect(screen, WHITE, enemy.rect())

        for bullet in self.player_bullets:
            pygame.draw.rect(screen, WHITE, bullet.rect())
        for bullet in self.enemy_bullets:
            pygame.draw.rect(screen, RED, bullet.rect())

        score_text = self.font.render(f"Score: {self.score}", True, WHITE)
        screen.blit(score_text, (10, 10))

        if self.game_over and not getattr(self, "_game_over_logged", False):
            # NOTE: no proper game-over screen yet - see Task 2 in the README.
            print("Game over! Final score:", self.score)
            self._game_over_logged = True
```

--- game/player.py ---

```
import pygame

class Player:
    def __init__(self, x, y, width, height):
        self.x = x
        self.y = y
        self.width = width
        self.height = height
        self.speed = 6

    def move(self, dx, screen_width):
        self.x += dx
        self.x = max(0, min(self.x, screen_width - self.width))

    def rect(self):
        return pygame.Rect(self.x, self.y, self.width, self.height)

    def center_x(self):
        return self.x + self.width // 2
```

--- game/enemy.py ---

```
import pygame

class Enemy:
    def __init__(self, x, y, width=40, height=30):
        self.x = x
        self.y = y
        self.width = width
        self.height = height
        self.alive = True

    def rect(self):
        return pygame.Rect(self.x, self.y, self.width, self.height)


class EnemyGrid:
    def __init__(self, screen_width, rows=4, cols=8, speed=1.5, drop_amount=20):
        self.screen_width = screen_width
        self.speed = speed
        self.drop_amount = drop_amount
        self.direction = 1  # 1 = moving right, -1 = moving left

        self.enemies = []
        spacing_x, spacing_y = 55, 45
        start_x, start_y = 40, 40
        for row in range(rows):
            for col in range(cols):
                x = start_x + col * spacing_x
                y = start_y + row * spacing_y
                self.enemies.append(Enemy(x, y))
```

```
    def alive_enemies(self):
        return [e for e in self.enemies if e.alive]

    def move(self):
        alive = self.alive_enemies()
        if not alive:
            return

        min_x = min(e.x for e in alive)
        max_x = max(e.x + e.width for e in alive)

        hit_edge = (self.direction == 1 and max_x + self.speed >= self.screen_width) or \
                   (self.direction == -1 and min_x - self.speed <= 0)

        if hit_edge:
            self.direction *= -1
            for e in alive:
                e.y += self.drop_amount
        else:
            for e in alive:
                e.x += self.speed * self.direction

    def reached_bottom(self, limit_y):
        return any(e.y + e.height >= limit_y for e in self.alive_enemies())
```

--- game/bullet.py ---

```
import pygame

class Bullet:
    def __init__(self, x, y, width=4, height=12, speed=8, direction=-1):
        self.x = x
        self.y = y
        self.width = width
        self.height = height
        self.speed = speed
        self.direction = direction  # -1 = moving up (player bullet), 1 = moving down (enemy bullet)

    def move(self):
        self.y += self.speed * self.direction

    def off_screen(self, screen_height):
        return self.y < 0 or self.y > screen_height

    def rect(self):
        return pygame.Rect(self.x, self.y, self.width, self.height)
```

Claude

I've read all five files and haven't changed anything.

How it works now

- **Game loop (main.py):** Each frame clears the screen, passes events to `engine.handle_event`, then calls `handle_input` (held keys, movement), `update` (all game logic), and `render`, at 60 FPS. Update and render are separate, which is good.
- **Bullets:** Space fires a player bullet (direction -1) from the player's center, with a 15-frame cooldown. Each enemy has a 1% chance per frame of firing a bullet downward (direction 1). Bullets move `speed * direction` per frame and are removed once `off_screen` is true.
- **Enemies:** `EnemyGrid` builds a 4x8 grid. Each frame `move()` shifts all living enemies sideways. When the group's edge would touch the screen edge, it reverses direction and drops every enemy by 20px. Dead enemies are only flagged `alive = False`, never removed.
- **Collisions:** Rect-based `colliderect`. Player bullet vs. enemy sets `enemy.alive = False`, removes the bullet and adds 1 to the score. Enemy bullet vs. player sets `game_over`. The game also ends if any enemy's bottom reaches the player's y.

Bugs and weak spots

1. **Removing from a list while iterating** (the one your comment flags). `self.player_bullets.remove(bullet)` inside `for bullet in self.player_bullets` skips the next bullet that frame, so a collision can be missed.
2. **No game-over screen.** The state only prints to the console, and nothing stops rendering or input handling meaningfully (the player can still move and shoot, though `update` returns early).
3. **No win condition.** Killing all enemies does nothing, and `move()` just returns early.
4. **Enemy fire rate scales badly.** The 1% chance is per enemy per frame, so with 32 enemies you get roughly 19 shots per second at the start, and the rate drops as enemies die. The firing rate depends on how many enemies are alive, and any alive enemy can fire, including ones blocked by others in front.
5. **Enemy speed never increases**, so difficulty stays flat as the grid thins out.
6. **Float positions stored in ints:** `speed` is 1.5, so `e.x` becomes a float, but `pygame.Rect` truncates it. Movement and collision can look slightly jittery.
7. **Edge check uses current positions:** `max_x + speed >= screen_width` works, but after the drop the group may sit right at the edge, and the check is recomputed each frame via full list scans (minor performance cost).
8. `reached_bottom(self.player.y)` triggers when an enemy's bottom touches the top of the player's row, which is fine, but enemies overlapping the player horizontally aren't otherwise checked.
9. **No restart option.** After game over the only way out is closing the window.
10. **Small code smells:**
    - `main.py` creates `engine` at module level, outside `main()`.
    - `_game_over_logged` is set via `getattr` rather than initialized in `__init__`.
    - `random` is imported but enemy firing has no cap on bullets on screen.
    - Bullet position `x` is hardcoded with `- 2` rather than derived from bullet width.
    - `SCREEN.fill(BLACK)` is done before the event loop, which is fine, but `BLACK` and the screen aren't shared with the engine.
    - `Bullet` and `Player` have no sprites or `Rect` caching, so every collision check builds new Rect objects (fine at this scale).

Ready for your next instruction.

User

When I run the game, I always get "Game over! Final score: 0" within 1-2 seconds, no matter what I do, so it's unplayable. Find the root cause (check how often enemies fire, and how many bullets that produces per second at 60 FPS) and fix it so enemies only occasionally return fire, as the README says. Make the fire rate a single clearly named setting that's easy to change later. At the default setting, aim for roughly 1 enemy bullet per second on average so the game is challenging but winnable. Give me the full updated files and explain what was wrong.

Claude

What was wrong

The cause is the enemy fire logic in `game/game_engine.py`. In `update()`, every living enemy rolled a 1% chance to fire on every frame:

```
for enemy in self.enemy_grid.alive_enemies():
    if random.random() < self.enemy_fire_chance:   # 0.01 each
```

A 1% chance per frame sounds small, but it is applied 32 times per frame (4x8 grid) at 60 FPS:

- 32 enemies x 0.01 = 0.32 bullets per frame
- 0.32 x 60 FPS = **about 19 enemy bullets per second**

Enemy bullets move 8px per frame and take roughly 1.2 seconds to reach the player's row. Within the first second or two there are 20-40 bullets in flight, spread across the whole grid width (about 425px). The player is about 44px wide to a bullet, so each bullet has roughly a 10% chance of lining up with the player. With that many bullets, a hit within 1-2 seconds is almost guaranteed, which is why you always got "Game over! Final score: 0".

The game also scaled badly: the fire rate depended on how many enemies were alive, so it dropped as you killed them.

The fix

The fire rate is now a single constant at the top of the file, `ENEMY_BULLETS_PER_SECOND`, set to `1.0`. Instead of every enemy rolling separately, the whole grid rolls once per frame with probability `ENEMY_BULLETS_PER_SECOND / FPS`. If the roll succeeds, one randomly chosen living enemy fires. This gives about 1 bullet per second on average, no matter how many enemies are alive. I also added a check for when no enemies are alive, because `random.choice` on an empty list would crash.

Only one file changes. `main.py`, `player.py`, `enemy.py` and `bullet.py` stay exactly as you have them.

game/game_engine.py

```
import pygame
import random
from .player import Player
from .enemy import EnemyGrid
from .bullet import Bullet

# Game Engine

WHITE = (255, 255, 255)
GREEN = (0, 200, 0)
RED = (220, 60, 60)

# ---------------------------------------------------------------
# SETTINGS - change these to tune the game
# ---------------------------------------------------------------
# Average number of bullets the ENEMIES fire per second (whole grid,
# not per enemy). 1.0 = about one enemy bullet per second.
# Lower = easier, higher = harder (e.g. 0.5 easy, 2.0 hard).
ENEMY_BULLETS_PER_SECOND = 1.0

# Must match the FPS used in main.py, because the fire chance below
# is calculated per frame.
FPS = 60


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.player = Player(width // 2 - 20, height - 50, 40, 20)
        self.enemy_grid = EnemyGrid(width)

        self.player_bullets = []
        self.enemy_bullets = []
        self._shoot_cooldown = 0

        # Chance (per frame) that the enemy grid fires ONE bullet.
        # Example: 1.0 bullet/sec / 60 frames/sec = about 1.7% per frame.
        self.enemy_fire_chance = ENEMY_BULLETS_PER_SECOND / FPS

        self.score = 0
        self.font = pygame.font.SysFont("Arial", 30)
        self.game_over = False

    def handle_event(self, event):
        if event.type == pygame.KEYDOWN and event.key == pygame.K_SPACE:
            if self._shoot_cooldown <= 0:
                bullet_x = self.player.center_x() - 2
                self.player_bullets.append(Bullet(bullet_x, self.player.y, direction=-1))
                self._shoot_cooldown = 15
```

```
def handle_input(self):
    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT] or keys[pygame.K_a]:
        self.player.move(-self.player.speed, self.width)
    if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
        self.player.move(self.player.speed, self.width)

def update(self):
    if self.game_over:
        return

    if self._shoot_cooldown > 0:
        self._shoot_cooldown -= 1

    self.enemy_grid.move()

    # Enemy fire: roll ONCE per frame for the whole grid (not once per
    # enemy). If the roll succeeds, one random living enemy shoots.
    # This keeps the fire rate steady no matter how many enemies are left.
    alive = self.enemy_grid.alive_enemies()
    if alive and random.random() < self.enemy_fire_chance:
        shooter = random.choice(alive)
        bullet_x = shooter.x + shooter.width // 2
        self.enemy_bullets.append(Bullet(bullet_x, shooter.y + shooter.height, direction=1))

    for bullet in self.player_bullets:
        bullet.move()
    for bullet in self.enemy_bullets:
        bullet.move()

    self.player_bullets = [b for b in self.player_bullets if not b.off_screen(self.height)]
    self.enemy_bullets = [b for b in self.enemy_bullets if not b.off_screen(self.height)]

    # NOTE: this removes a bullet from player_bullets while iterating
    # directly over that same list. Python skips the element right
    # after a removed one, so when two enemies are hit on the same
    # frame the second collision can be missed - the bullet appears
    # to pass straight through. See Task 1 in the README.
    for bullet in self.player_bullets:
        for enemy in self.enemy_grid.alive_enemies():
            if bullet.rect().colliderect(enemy.rect()):
                enemy.alive = False
                self.player_bullets.remove(bullet)
                self.score += 1
                break

    for bullet in self.enemy_bullets:
        if bullet.rect().colliderect(self.player.rect()):
            self.game_over = True
            break

    if self.enemy_grid.reached_bottom(self.player.y):
        self.game_over = True

def render(self, screen):
    pygame.draw.rect(screen, GREEN, self.player.rect())

    for enemy in self.enemy_grid.alive_enemies():
        pygame.draw.rect(screen, WHITE, enemy.rect())

    for bullet in self.player_bullets:
        pygame.draw.rect(screen, WHITE, bullet.rect())
    for bullet in self.enemy_bullets:
        pygame.draw.rect(screen, RED, bullet.rect())

    score_text = self.font.render(f"Score: {self.score}", True, WHITE)
    screen.blit(score_text, (10, 10))

    if self.game_over and not getattr(self, "_game_over_logged", False):
        # NOTE: no proper game-over screen yet - see Task 2 in the README.
        print("Game over! Final score:", self.score)
        self._game_over_logged = True
```

To change the difficulty later, edit only `ENEMY_BULLETS_PER_SECOND` at the top of the file. If you ever change `FPS` in `main.py`, change the `FPS` constant here too, so the per-second rate stays correct.

Everything else is untouched, including the remove-while-iterating collision bug and the missing game-over screen, which I'm leaving for your next instruction. Any enemy can still fire, even one with another enemy directly in front of it. I kept it that way to stay simple, but restricting fire to the front enemy in each column is an easy follow-up if you want it.

User

Task 1: Bullets sometimes pass straight through enemies without registering a hit, especially when firing rapidly or when two enemies are hit close together. Find the root cause (look at how bullets are removed from the list while it is being iterated over, and at whether a bullet can skip past an enemy between frames). Fix it so that: (1) each bullet destroys at most one enemy and is removed safely, (2) two enemies hit in the same frame are both handled correctly, (3) a bullet that crosses an enemy between two frames still counts as a hit, and (4) enemy bullets hitting the player still work. Explain briefly what was wrong, then give the full updated code for the changed files.

Claude

What was wrong

1. Removing from a list while looping over it (the main bug). In `game_engine.py`, `self.player_bullets.remove(bullet)` ran inside `for bullet in self.player_bullets`. When Python removes an item, the next item shifts into its slot and the loop steps past it, so the next bullet in the list got no collision check that frame. Two bullets hitting two enemies on the same frame meant the second hit was skipped. Mostly that only delays the hit by a frame, but it is still a real bug, and it is what makes bullets look like they pass through.

2. Collision only checked the bullet's current position. A bullet moves 8px per frame, but only the rectangle where it ended up was tested. With today's numbers (an 8px step, a 12px bullet and a 30px enemy) a bullet can't fully jump an enemy. But any increase in bullet speed, or an enemy row dropping 20px onto a bullet, makes skipping possible. The fix tests the whole path the bullet travelled this frame.

3. A smaller order problem. Off-screen bullets were deleted before collision checks, so a bullet that hit something and left the screen on the same frame lost its hit. Collisions now run first, then off-screen bullets are removed.

The fix

- `bullet.py`**:** each bullet remembers its previous `y`, and a new `swept_rect()` returns a rectangle covering everything the bullet passed through this frame.
- `game_engine.py`**:**

- The collision loop builds a new list of surviving bullets instead of removing from the live list, so nothing is skipped. - Each bullet destroys at most one enemy. If its path touches two, it takes the lowest one, which it reached first. - Enemies are re-checked per bullet, so two bullets can never both score on the same enemy. - Enemy bullets use the same path check against the player. - The fire-rate fix from earlier is kept.

`main.py`, `player.py` and `enemy.py` are unchanged.

game/bullet.py

```
import pygame

class Bullet:
    def __init__(self, x, y, width=4, height=12, speed=8, direction=-1):
        self.x = x
        self.y = y
        # Where the bullet was on the previous frame. Used to build the
        # "swept" rectangle so fast bullets can't skip over a target.
        self.prev_y = y
        self.width = width
        self.height = height
        self.speed = speed
```

```
        self.direction = direction  # -1 = moving up (player bullet), 1 = moving down (enemy bullet)

    def move(self):
        # Remember the old position BEFORE moving.
        self.prev_y = self.y
        self.y += self.speed * self.direction

    def off_screen(self, screen_height):
        return self.y < 0 or self.y > screen_height

    def rect(self):
        # Where the bullet is right now (used for drawing).
        return pygame.Rect(self.x, self.y, self.width, self.height)

    def swept_rect(self):
        # One rectangle covering everything the bullet passed through
        # between the previous frame and this frame (old spot + new spot).
        top = min(self.prev_y, self.y)
        total_height = abs(self.y - self.prev_y) + self.height
        return pygame.Rect(self.x, top, self.width, total_height)
```

game/game_engine.py

```
import pygame
import random
from .player import Player
from .enemy import EnemyGrid
from .bullet import Bullet

# Game Engine

WHITE = (255, 255, 255)
GREEN = (0, 200, 0)
RED = (220, 60, 60)

# ------------------------------------------------------------
# SETTINGS - change these to tune the game
# ------------------------------------------------------------
# Average number of bullets the ENEMIES fire per second (whole grid,
# not per enemy). 1.0 = about one enemy bullet per second.
# Lower = easier, higher = harder (e.g. 0.5 easy, 2.0 hard).
ENEMY_BULLETS_PER_SECOND = 1.0

# Must match the FPS used in main.py, because the fire chance below
# is calculated per frame.
FPS = 60


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.player = Player(width // 2 - 20, height - 50, 40, 20)
        self.enemy_grid = EnemyGrid(width)

        self.player_bullets = []
        self.enemy_bullets = []
        self._shoot_cooldown = 0

        # Chance (per frame) that the enemy grid fires ONE bullet.
        # Example: 1.0 bullet/sec / 60 frames/sec = about 1.7% per frame.
        self.enemy_fire_chance = ENEMY_BULLETS_PER_SECOND / FPS

        self.score = 0
        self.font = pygame.font.SysFont("Arial", 30)
        self.game_over = False

    def handle_event(self, event):
        if event.type == pygame.KEYDOWN and event.key == pygame.K_SPACE:
            if self._shoot_cooldown <= 0:
                bullet_x = self.player.center_x() - 2
                self.player_bullets.append(Bullet(bullet_x, self.player.y, direction=-1))
                self._shoot_cooldown = 15
```

```
def handle_input(self):
    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT] or keys[pygame.K_a]:
        self.player.move(-self.player.speed, self.width)
    if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
        self.player.move(self.player.speed, self.width)

def update(self):
    if self.game_over:
        return

    if self._shoot_cooldown > 0:
        self._shoot_cooldown -= 1

    self.enemy_grid.move()

    # Enemy fire: roll ONCE per frame for the whole grid (not once per
    # enemy). If the roll succeeds, one random living enemy shoots.
    # This keeps the fire rate steady no matter how many enemies are left.
    alive = self.enemy_grid.alive_enemies()
    if alive and random.random() < self.enemy_fire_chance:
        shooter = random.choice(alive)
        bullet_x = shooter.x + shooter.width // 2
        self.enemy_bullets.append(Bullet(bullet_x, shooter.y + shooter.height, direction=1))

    # Move every bullet (each one remembers where it was last frame).
    for bullet in self.player_bullets:
        bullet.move()
    for bullet in self.enemy_bullets:
        bullet.move()

    # --- Player bullets vs enemies ---
    # We build a NEW list of bullets that survive instead of removing
    # items from the list we are looping over (that caused skipped checks).
    surviving_bullets = []
    for bullet in self.player_bullets:
        path = bullet.swept_rect()  # everything the bullet crossed this frame
        hit_enemy = None

        # alive_enemies() is called fresh for every bullet, so an enemy
        # killed by an earlier bullet this frame can't be hit again.
        for enemy in self.enemy_grid.alive_enemies():
            if path.colliderect(enemy.rect()):
                # If the path touches more than one enemy, take the
                # lowest one (largest y) - the bullet reached it first.
                if hit_enemy is None or enemy.y > hit_enemy.y:
                    hit_enemy = enemy

        if hit_enemy is not None:
            hit_enemy.alive = False   # at most ONE enemy per bullet
            self.score += 1           # the bullet is used up (not kept)
        else:
            surviving_bullets.append(bullet)
    self.player_bullets = surviving_bullets

    # --- Enemy bullets vs player ---
    # Same swept check, so a fast bullet can't skip over the player.
    for bullet in self.enemy_bullets:
        if bullet.swept_rect().colliderect(self.player.rect()):
            self.game_over = True
            break

    # Remove off-screen bullets AFTER collisions, so a bullet that hit
    # something and left the screen on the same frame still counts.
    self.player_bullets = [b for b in self.player_bullets if not b.off_screen(self.height)]
    self.enemy_bullets = [b for b in self.enemy_bullets if not b.off_screen(self.height)]

    if self.enemy_grid.reached_bottom(self.player.y):
        self.game_over = True

def render(self, screen):
    pygame.draw.rect(screen, GREEN, self.player.rect())

    for enemy in self.enemy_grid.alive_enemies():
```

```
            pygame.draw.rect(screen, WHITE, enemy.rect())

        for bullet in self.player_bullets:
            pygame.draw.rect(screen, WHITE, bullet.rect())
        for bullet in self.enemy_bullets:
            pygame.draw.rect(screen, RED, bullet.rect())

        score_text = self.font.render(f"Score: {self.score}", True, WHITE)
        screen.blit(score_text, (10, 10))

        if self.game_over and not getattr(self, "_game_over_logged", False):
            # NOTE: no proper game-over screen yet - see Task 2 in the README.
            print("Game over! Final score:", self.score)
            self._game_over_logged = True
```

One limit remains: the sweep only covers the bullet's own movement. An enemy row dropping 20px onto a bullet in one frame is still safe today (the combined 28px of relative movement is less than the 42px a bullet and enemy need to pass fully through each other). If you ever raise the drop distance a lot, the enemy's movement would need sweeping too.

Ready for Task 2 whenever you are.

User

Task 2: Implement a proper Game Over condition. The game ends when the player is hit by an enemy bullet OR the enemy grid reaches the player's row at the bottom. Instead of printing to the console, draw a Game Over screen in the Pygame window showing "GAME OVER", the final score, and "Press any key to continue". It must wait for input without freezing, and closing the window must still quit cleanly. For now, pressing a key can simply close the game (I'll add replay in the next task). Keep the existing game loop structure and give me the full updated code for every changed file.

Claude

Only two files change: `main.py` and `game/game_engine.py`. `player.py`, `enemy.py` and `bullet.py` stay as they are.

What changed

- **Game-over conditions:** the two conditions (enemy bullet hits the player, or the grid reaches the player's row) were already in `update()`. I kept them as they are.
- **Game Over screen:** the console `print` and the `_game_over_logged` flag are gone. `render()` now draws the frozen game with a dark overlay, "GAME OVER", the final score, and "Press any key to continue".
- **No freezing:** the main loop keeps running while the game is over. It still handles events and redraws every frame, and only `update()` skips the game logic. So closing the window works at any time.
- **Quitting on a key press:** the engine has a new `quit_requested` flag. A key press on the game-over screen sets it, and `main.py` checks it after the event loop and ends the loop. `pygame.quit()` then runs as before.
- **Short input delay (my addition):** keys are ignored for about 0.75 seconds after game over, and the prompt only appears once keys are accepted. Without this, a player still mashing Space or the arrow keys would dismiss the screen before seeing their score. Change `GAME_OVER_INPUT_DELAY_SECONDS` to adjust it, or set it to `0` to remove it.
- **Input after game over:** the player can no longer move or shoot, so the ship doesn't slide around under the overlay.

main.py

```
import pygame
from game.game_engine import GameEngine

# Initialize pygame/Start application
pygame.init()

# Screen dimensions
WIDTH, HEIGHT = 600, 700
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Space Invaders - Pygame Version")
```

```
# Colors
BLACK = (0, 0, 0)

# Clock
clock = pygame.time.Clock()
FPS = 60

# Game loop
engine = GameEngine(WIDTH, HEIGHT)

def main():
    running = True
    while running:
        SCREEN.fill(BLACK)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            engine.handle_event(event)

        # The engine asks us to quit (e.g. a key was pressed on the
        # Game Over screen). Ending the loop lets pygame.quit() run below.
        if engine.quit_requested:
            running = False

        engine.handle_input()
        engine.update()
        engine.render(SCREEN)

        pygame.display.flip()
        clock.tick(FPS)

    pygame.quit()

if __name__ == "__main__":
    main()
```

game/game_engine.py

```
import pygame
import random
from .player import Player
from .enemy import EnemyGrid
from .bullet import Bullet

# Game Engine

WHITE = (255, 255, 255)
GREEN = (0, 200, 0)
RED = (220, 60, 60)

# ---------------------------------------------------------------
# SETTINGS - change these to tune the game
# ---------------------------------------------------------------
# Average number of bullets the ENEMIES fire per second (whole grid,
# not per enemy). 1.0 = about one enemy bullet per second.
# Lower = easier, higher = harder (e.g. 0.5 easy, 2.0 hard).
ENEMY_BULLETS_PER_SECOND = 1.0

# Must match the FPS used in main.py, because the fire chance below
# is calculated per frame.
FPS = 60

# After game over, key presses are ignored for this long so a player who
# is still mashing keys doesn't skip the screen by accident.
# Set to 0 to accept keys immediately.
GAME_OVER_INPUT_DELAY_SECONDS = 0.75


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.player = Player(width // 2 - 20, height - 50, 40, 20)
```

```
        self.enemy_grid = EnemyGrid(width)

        self.player_bullets = []
        self.enemy_bullets = []
        self._shoot_cooldown = 0

        # Chance (per frame) that the enemy grid fires ONE bullet.
        # Example: 1.0 bullet/sec / 60 frames/sec = about 1.7% per frame.
        self.enemy_fire_chance = ENEMY_BULLETS_PER_SECOND / FPS

        self.score = 0
        self.game_over = False

        # Fonts: normal (score), big (GAME OVER title), small (prompt)
        self.font = pygame.font.SysFont("Arial", 30)
        self.title_font = pygame.font.SysFont("Arial", 64, bold=True)
        self.small_font = pygame.font.SysFont("Arial", 24)

        # Game Over state
        self._game_over_timer = 0   # frames since the game ended
        self._game_over_delay = int(GAME_OVER_INPUT_DELAY_SECONDS * FPS)
        self.quit_requested = False  # main.py checks this to close the game

    def handle_event(self, event):
        # On the Game Over screen, any key press asks the game to quit
        # (but only after the short input delay has passed).
        if self.game_over:
            if event.type == pygame.KEYDOWN and self._game_over_timer >= self._game_over_delay:
                self.quit_requested = True
            return  # no shooting while the game is over

        if event.type == pygame.KEYDOWN and event.key == pygame.K_SPACE:
            if self._shoot_cooldown <= 0:
                bullet_x = self.player.center_x() - 2
                self.player_bullets.append(Bullet(bullet_x, self.player.y, direction=-1))
                self._shoot_cooldown = 15

    def handle_input(self):
        if self.game_over:
            return  # the player can't move once the game is over

        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT] or keys[pygame.K_a]:
            self.player.move(-self.player.speed, self.width)
        if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
            self.player.move(self.player.speed, self.width)

    def update(self):
        if self.game_over:
            # The game logic is stopped, but we keep counting frames so
            # the input delay on the Game Over screen can finish.
            self._game_over_timer += 1
            return

        if self._shoot_cooldown > 0:
            self._shoot_cooldown -= 1

        self.enemy_grid.move()

        # Enemy fire: roll ONCE per frame for the whole grid (not once per
        # enemy). If the roll succeeds, one random living enemy shoots.
        # This keeps the fire rate steady no matter how many enemies are left.
        alive = self.enemy_grid.alive_enemies()
        if alive and random.random() < self.enemy_fire_chance:
            shooter = random.choice(alive)
            bullet_x = shooter.x + shooter.width // 2
            self.enemy_bullets.append(Bullet(bullet_x, shooter.y + shooter.height, direction=1))

        # Move every bullet (each one remembers where it was last frame).
        for bullet in self.player_bullets:
            bullet.move()
        for bullet in self.enemy_bullets:
            bullet.move()
```

```
        # --- Player bullets vs enemies ---
        # We build a NEW list of bullets that survive instead of removing
        # items from the list we are looping over (that caused skipped checks).
        surviving_bullets = []
        for bullet in self.player_bullets:
            path = bullet.swept_rect()  # everything the bullet crossed this frame
            hit_enemy = None

            # alive_enemies() is called fresh for every bullet, so an enemy
            # killed by an earlier bullet this frame can't be hit again.
            for enemy in self.enemy_grid.alive_enemies():
                if path.colliderect(enemy.rect()):
                    # If the path touches more than one enemy, take the
                    # lowest one (largest y) - the bullet reached it first.
                    if hit_enemy is None or enemy.y > hit_enemy.y:
                        hit_enemy = enemy

            if hit_enemy is not None:
                hit_enemy.alive = False   # at most ONE enemy per bullet
                self.score += 1           # the bullet is used up (not kept)
            else:
                surviving_bullets.append(bullet)
        self.player_bullets = surviving_bullets

        # --- GAME OVER condition 1: an enemy bullet hits the player ---
        # Same swept check, so a fast bullet can't skip over the player.
        for bullet in self.enemy_bullets:
            if bullet.swept_rect().colliderect(self.player.rect()):
                self.game_over = True
                break

        # Remove off-screen bullets AFTER collisions, so a bullet that hit
        # something and left the screen on the same frame still counts.
        self.player_bullets = [b for b in self.player_bullets if not b.off_screen(self.height)]
        self.enemy_bullets = [b for b in self.enemy_bullets if not b.off_screen(self.height)]

        # --- GAME OVER condition 2: the enemies reach the player's row ---
        if self.enemy_grid.reached_bottom(self.player.y):
            self.game_over = True

    def render(self, screen):
        # Draw the game as usual (when the game is over it stays frozen
        # on screen behind the Game Over overlay).
        pygame.draw.rect(screen, GREEN, self.player.rect())

        for enemy in self.enemy_grid.alive_enemies():
            pygame.draw.rect(screen, WHITE, enemy.rect())

        for bullet in self.player_bullets:
            pygame.draw.rect(screen, WHITE, bullet.rect())
        for bullet in self.enemy_bullets:
            pygame.draw.rect(screen, RED, bullet.rect())

        score_text = self.font.render(f"Score: {self.score}", True, WHITE)
        screen.blit(score_text, (10, 10))

        if self.game_over:
            self._draw_game_over(screen)

    def _draw_game_over(self, screen):
        # Semi-transparent dark layer so the text is easy to read.
        overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
        overlay.fill((0, 0, 0, 180))  # last number = transparency (0-255)
        screen.blit(overlay, (0, 0))

        center_x = self.width // 2
        center_y = self.height // 2

        # "GAME OVER" title
        title = self.title_font.render("GAME OVER", True, RED)
        screen.blit(title, title.get_rect(center=(center_x, center_y - 60)))

        # Final score
        final_score = self.font.render(f"Final Score: {self.score}", True, WHITE)
```

```
        screen.blit(final_score, final_score.get_rect(center=(center_x, center_y + 10)))

        # Prompt - only shown once key presses are being accepted
        if self._game_over_timer >= self._game_over_delay:
            prompt = self.small_font.render("Press any key to continue", True, WHITE)
            screen.blit(prompt, prompt.get_rect(center=(center_x, center_y + 70)))
```

To test it, stand still until an enemy bullet hits you, or let the grid drop to your row. The overlay should appear, and after about 0.75 seconds the prompt shows and any key closes the game. Clicking the window's close button also works at every point.

Ready for the replay task whenever you are. `quit_requested` is the spot to change: instead of quitting, that key press can reset the engine.

User

Task 3: After the Game Over screen, let the player choose "Play again" or "Exit". If they play again, show a difficulty menu with Easy, Medium and Hard (select with keys 1/2/3, or Up/Down + Enter). Difficulty must change both the enemy movement speed and the enemy fire rate (Easy slowest/least firing, Hard fastest/most firing), reusing the fire-rate setting we added earlier. Define the settings in one dictionary so they're easy to tweak. Starting a new game must fully reset the score, player, enemies, bullets and game-over state. Exit (Esc or Q) must quit cleanly. Give me the full updated code for every changed file.

Claude

Only one file changes: `game/game_engine.py`. `main.py` already quits when `engine.quit_requested` is set, and `EnemyGrid` already accepts a `speed` argument, so `main.py`, `player.py`, `enemy.py` and `bullet.py` stay as they are.

What changed

- **Difficulty settings:** one dictionary, `DIFFICULTY_SETTINGS`, holds the enemy speed and `enemy_bullets_per_second` for Easy, Medium and Hard. This replaces the old `ENEMY_BULLETS_PER_SECOND` constant. It is the same fire-rate setting, now stored per difficulty and still converted to a per-frame chance by dividing by `FPS`. Medium uses the old defaults (speed 1.5, 1 bullet per second), so the game plays as before.
- **Screens:** the engine now has three states: playing, game over and difficulty menu.
- **Game Over screen:** it shows "Play again" and "Exit". Use Up/Down and Enter to choose. The 0.75 s input delay from Task 2 still applies, so mashing keys can't skip it.
- **Difficulty menu:** press 1, 2 or 3 to start straight away, or use Up/Down and Enter. The highlight starts on Medium.
- **Exit:** Esc or Q on the Game Over screen or the menu quits cleanly, as does choosing "Exit". Closing the window still works at any point.
- **Full reset:** a new method, `start_new_game(difficulty)`, rebuilds the player, enemy grid, bullets, score, cooldown, game-over state and timers. `__init__` calls it too, so the first game and every replay start identically.
- **First launch:** the game starts directly on Medium, with no menu. The menu only appears after Game Over, as you described.
- **HUD:** the current difficulty is shown top right.

game/game_engine.py

```
import pygame
import random
from .player import Player
from .enemy import EnemyGrid
from .bullet import Bullet

# Game Engine

WHITE = (255, 255, 255)
GREEN = (0, 200, 0)
```

```
RED = (220, 60, 60)

# ------------------------------------------------------------
# SETTINGS - change these to tune the game
# ------------------------------------------------------------
# Must match the FPS used in main.py, because the fire chance is
# calculated per frame.
FPS = 60

# After game over, key presses are ignored for this long so a player who
# is still mashing keys doesn't skip the screen by accident.
# Set to 0 to accept keys immediately.
GAME_OVER_INPUT_DELAY_SECONDS = 0.75

# Difficulty settings - all in ONE place, easy to tweak.
#   enemy_speed              : pixels the enemy grid moves per frame
#   enemy_bullets_per_second : average bullets the WHOLE grid fires per second
# The order here is the order shown in the menu (keys 1, 2, 3).
DIFFICULTY_SETTINGS = {
    "Easy":   {"enemy_speed": 1.0, "enemy_bullets_per_second": 0.5},
    "Medium": {"enemy_speed": 1.5, "enemy_bullets_per_second": 1.0},
    "Hard":   {"enemy_speed": 2.5, "enemy_bullets_per_second": 2.0},
}
DEFAULT_DIFFICULTY = "Medium"

# Game states
STATE_PLAYING = "playing"
STATE_GAME_OVER = "game_over"
STATE_MENU = "menu"

# Choices on the Game Over screen
GAME_OVER_OPTIONS = ["Play again", "Exit"]


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        # Fonts are created once: normal, big (titles), small (hints)
        self.font = pygame.font.SysFont("Arial", 30)
        self.title_font = pygame.font.SysFont("Arial", 64, bold=True)
        self.small_font = pygame.font.SysFont("Arial", 24)

        # Names of the difficulties in menu order: ["Easy", "Medium", "Hard"]
        self.difficulty_names = list(DIFFICULTY_SETTINGS.keys())

        self.quit_requested = False  # main.py checks this to close the game

        # Set up everything for the first game.
        self.start_new_game(DEFAULT_DIFFICULTY)

    def start_new_game(self, difficulty):
        """Reset EVERYTHING and start a fresh game on the given difficulty."""
        settings = DIFFICULTY_SETTINGS[difficulty]
        self.difficulty = difficulty

        # Fresh player and enemies (enemy speed comes from the difficulty)
        self.player = Player(self.width // 2 - 20, self.height - 50, 40, 20)
        self.enemy_grid = EnemyGrid(self.width, speed=settings["enemy_speed"])

        # Empty the bullets and reset the shooting cooldown
        self.player_bullets = []
        self.enemy_bullets = []
        self._shoot_cooldown = 0

        # Chance (per frame) that the enemy grid fires ONE bullet.
        # Example: 1.0 bullet/sec / 60 frames/sec = about 1.7% per frame.
        self.enemy_fire_chance = settings["enemy_bullets_per_second"] / FPS

        # Score and state
        self.score = 0
        self.state = STATE_PLAYING
```

```
    # Game Over screen state
    self._game_over_timer = 0   # frames since the game ended
    self._game_over_delay = int(GAME_OVER_INPUT_DELAY_SECONDS * FPS)
    self._game_over_selected = 0   # 0 = Play again, 1 = Exit

    # Menu state (highlight starts on the default difficulty)
    self._menu_selected = self.difficulty_names.index(DEFAULT_DIFFICULTY)

def _end_game(self):
    """Switch to the Game Over screen."""
    self.state = STATE_GAME_OVER
    self._game_over_timer = 0
    self._game_over_selected = 0

# ------------------------------------------------------------
# Input
# ------------------------------------------------------------
def handle_event(self, event):
    # Only key presses matter here (window close is handled in main.py)
    if event.type != pygame.KEYDOWN:
        return

    if self.state == STATE_MENU:
        self._handle_menu_key(event.key)
    elif self.state == STATE_GAME_OVER:
        self._handle_game_over_key(event.key)
    else:
        # Playing: Space shoots
        if event.key == pygame.K_SPACE and self._shoot_cooldown <= 0:
            bullet_x = self.player.center_x() - 2
            self.player_bullets.append(Bullet(bullet_x, self.player.y, direction=-1))
            self._shoot_cooldown = 15

def _handle_game_over_key(self, key):
    # Esc or Q always quits (even during the short input delay).
    if key in (pygame.K_ESCAPE, pygame.K_q):
        self.quit_requested = True
        return

    # Ignore other keys until the delay has passed.
    if self._game_over_timer < self._game_over_delay:
        return

    if key == pygame.K_UP:
        self._game_over_selected = (self._game_over_selected - 1) % len(GAME_OVER_OPTIONS)
    elif key == pygame.K_DOWN:
        self._game_over_selected = (self._game_over_selected + 1) % len(GAME_OVER_OPTIONS)
    elif key in (pygame.K_RETURN, pygame.K_KP_ENTER):
        if self._game_over_selected == 0:
            self.state = STATE_MENU          # Play again -> difficulty menu
        else:
            self.quit_requested = True       # Exit

def _handle_menu_key(self, key):
    # Esc or Q quits from the menu too.
    if key in (pygame.K_ESCAPE, pygame.K_q):
        self.quit_requested = True
        return

    # Keys 1, 2, 3 pick a difficulty straight away.
    # (pygame.K_1, K_2, K_3 are consecutive numbers.)
    number_index = key - pygame.K_1
    if 0 <= number_index < len(self.difficulty_names):
        self.start_new_game(self.difficulty_names[number_index])
        return

    # Up/Down moves the highlight, Enter confirms.
    if key == pygame.K_UP:
        self._menu_selected = (self._menu_selected - 1) % len(self.difficulty_names)
    elif key == pygame.K_DOWN:
        self._menu_selected = (self._menu_selected + 1) % len(self.difficulty_names)
    elif key in (pygame.K_RETURN, pygame.K_KP_ENTER):
        self.start_new_game(self.difficulty_names[self._menu_selected])
```

```
def handle_input(self):
    if self.state != STATE_PLAYING:
        return  # the player can only move while playing

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT] or keys[pygame.K_a]:
        self.player.move(-self.player.speed, self.width)
    if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
        self.player.move(self.player.speed, self.width)

# ------------------------------------------------------------
# Game logic
# ------------------------------------------------------------
def update(self):
    if self.state == STATE_GAME_OVER:
        # The game logic is stopped, but we keep counting frames so
        # the input delay on the Game Over screen can finish.
        self._game_over_timer += 1
        return
    if self.state != STATE_PLAYING:
        return  # nothing to update in the menu

    if self._shoot_cooldown > 0:
        self._shoot_cooldown -= 1

    self.enemy_grid.move()

    # Enemy fire: roll ONCE per frame for the whole grid (not once per
    # enemy). If the roll succeeds, one random living enemy shoots.
    # This keeps the fire rate steady no matter how many enemies are left.
    alive = self.enemy_grid.alive_enemies()
    if alive and random.random() < self.enemy_fire_chance:
        shooter = random.choice(alive)
        bullet_x = shooter.x + shooter.width // 2
        self.enemy_bullets.append(Bullet(bullet_x, shooter.y + shooter.height, direction=1))

    # Move every bullet (each one remembers where it was last frame).
    for bullet in self.player_bullets:
        bullet.move()
    for bullet in self.enemy_bullets:
        bullet.move()

    # --- Player bullets vs enemies ---
    # We build a NEW list of bullets that survive instead of removing
    # items from the list we are looping over (that caused skipped checks).
    surviving_bullets = []
    for bullet in self.player_bullets:
        path = bullet.swept_rect()  # everything the bullet crossed this frame
        hit_enemy = None

        # alive_enemies() is called fresh for every bullet, so an enemy
        # killed by an earlier bullet this frame can't be hit again.
        for enemy in self.enemy_grid.alive_enemies():
            if path.colliderect(enemy.rect()):
                # If the path touches more than one enemy, take the
                # lowest one (largest y) - the bullet reached it first.
                if hit_enemy is None or enemy.y > hit_enemy.y:
                    hit_enemy = enemy

        if hit_enemy is not None:
            hit_enemy.alive = False   # at most ONE enemy per bullet
            self.score += 1           # the bullet is used up (not kept)
        else:
            surviving_bullets.append(bullet)
    self.player_bullets = surviving_bullets

    # --- GAME OVER condition 1: an enemy bullet hits the player ---
    # Same swept check, so a fast bullet can't skip over the player.
    for bullet in self.enemy_bullets:
        if bullet.swept_rect().colliderect(self.player.rect()):
            self._end_game()
            break

    # Remove off-screen bullets AFTER collisions, so a bullet that hit
```

```
        # something and left the screen on the same frame still counts.
        self.player_bullets = [b for b in self.player_bullets if not b.off_screen(self.height)]
        self.enemy_bullets = [b for b in self.enemy_bullets if not b.off_screen(self.height)]

        # --- GAME OVER condition 2: the enemies reach the player's row ---
        if self.enemy_grid.reached_bottom(self.player.y):
            self._end_game()

    # ------------------------------------------------------------
    # Drawing
    # ------------------------------------------------------------
    def render(self, screen):
        # The difficulty menu has its own screen (no game behind it).
        if self.state == STATE_MENU:
            self._draw_menu(screen)
            return

        # Draw the game as usual (when the game is over it stays frozen
        # on screen behind the Game Over overlay).
        pygame.draw.rect(screen, GREEN, self.player.rect())

        for enemy in self.enemy_grid.alive_enemies():
            pygame.draw.rect(screen, WHITE, enemy.rect())

        for bullet in self.player_bullets:
            pygame.draw.rect(screen, WHITE, bullet.rect())
        for bullet in self.enemy_bullets:
            pygame.draw.rect(screen, RED, bullet.rect())

        score_text = self.font.render(f"Score: {self.score}", True, WHITE)
        screen.blit(score_text, (10, 10))

        # Current difficulty, top right
        level_text = self.small_font.render(self.difficulty, True, WHITE)
        screen.blit(level_text, level_text.get_rect(topright=(self.width - 10, 14)))

        if self.state == STATE_GAME_OVER:
            self._draw_game_over(screen)

    def _draw_options(self, screen, labels, selected, first_y, gap):
        """Draw a centered list of options; the selected one is green with a box."""
        center_x = self.width // 2
        for i, label in enumerate(labels):
            is_selected = (i == selected)
            color = GREEN if is_selected else WHITE
            text = self.font.render(label, True, color)
            rect = text.get_rect(center=(center_x, first_y + i * gap))
            screen.blit(text, rect)
            if is_selected:
                pygame.draw.rect(screen, GREEN, rect.inflate(40, 16), 2)

    def _draw_game_over(self, screen):
        # Semi-transparent dark layer so the text is easy to read.
        overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
        overlay.fill((0, 0, 0, 180))  # last number = transparency (0-255)
        screen.blit(overlay, (0, 0))

        center_x = self.width // 2
        center_y = self.height // 2

        # "GAME OVER" title
        title = self.title_font.render("GAME OVER", True, RED)
        screen.blit(title, title.get_rect(center=(center_x, center_y - 100)))

        # Final score
        final_score = self.font.render(f"Final Score: {self.score}", True, WHITE)
        screen.blit(final_score, final_score.get_rect(center=(center_x, center_y - 30)))

        # Options - only shown once key presses are being accepted
        if self._game_over_timer >= self._game_over_delay:
            self._draw_options(screen, GAME_OVER_OPTIONS, self._game_over_selected,
                               first_y=center_y + 40, gap=60)
            hint = self.small_font.render("Up/Down + Enter to choose  |  Esc / Q to exit", True, WHITE)
            screen.blit(hint, hint.get_rect(center=(center_x, center_y + 170)))
```

```
    def _draw_menu(self, screen):
        center_x = self.width // 2

        title = self.title_font.render("DIFFICULTY", True, GREEN)
        screen.blit(title, title.get_rect(center=(center_x, 180)))

        # "1. Easy", "2. Medium", "3. Hard"
        labels = [f"{i + 1}. {name}" for i, name in enumerate(self.difficulty_names)]
        self._draw_options(screen, labels, self._menu_selected, first_y=300, gap=70)

        hint = self.small_font.render("Press 1/2/3, or Up/Down + Enter  |  Esc / Q to exit", True, WHITE)
        screen.blit(hint, hint.get_rect(center=(center_x, 560)))
```

To test it, let the game end and wait for the options to appear. Then choose "Play again" and pick a difficulty. Score, enemies, bullets and the player position should all start fresh. On Hard the grid should visibly move faster and fire more often than on Easy.

The game still has no win condition, so clearing all enemies does nothing. That's a natural next task if your README includes it.

User

Task 4: Add basic sound effects using pygame.mixer for three events: the player firing, an enemy being destroyed, and game over (play the game-over sound once, not every frame). I have no audio files, so write a small standalone script (generate_sounds.py, standard library only: wave, struct, math) that creates short .wav files in an assets/sounds folder, and tell me how to run it once. Load the sounds in the game using paths that work no matter which folder I run the game from, and wrap the loading in try/except so the game still runs if audio isn't available. Give me the full updated code for every changed file.

Claude

Two files change: `game/game_engine.py` (updated) and `generate_sounds.py` (new, in the project root next to `main.py`). `main.py`, `player.py`, `enemy.py` and `bullet.py` are unchanged.

How to run the generator once

From the project root, run `python generate_sounds.py`. It works from any folder, because it saves next to its own location. It creates three files in `assets/sounds/`:

```
your_project/
|-- main.py
|-- generate_sounds.py        <- new
|-- assets/
|   `-- sounds/
|       |-- fire.wav          <- created by the script
|       |-- explosion.wav
|       `-- game_over.wav
`-- game/
    |-- game_engine.py        <- updated
    `-- ...
```

What changed

- **Paths:** the engine builds the sound path from the location of `game_engine.py` itself (`__file__`), not the current working directory, so it works no matter where you launch the game from.
- **Safe loading:** each sound loads in its own `try/except`, so a missing file or a machine with no audio device never crashes the game. The mixer is initialised in a `try/except` too, since `pygame.init()` doesn't raise an error when audio is unavailable. If something fails, you get one printed warning and the game runs silently.
- **Fire sound:** plays when a bullet is actually fired, so it respects the shoot cooldown.
- **Explosion sound:** plays when an enemy is destroyed.

- **Game over sound:** plays inside `_end_game()`, which runs once when the state changes, not every frame. I added a guard at the top of `_end_game()`. Without it, the sound could play twice in the rare frame where a bullet hit and the grid reaching the bottom both end the game.
- **Replay:** sounds are loaded once in `__init__`, not on every new game, so replaying doesn't reload files.

generate_sounds.py

```
# Creates three short .wav sound effects in assets/sounds/.
# Standard library only (wave, struct, math). Run it once:
#     python generate_sounds.py

import math
import os
import struct
import wave

SAMPLE_RATE = 22050  # samples per second (fine for simple effects)

# Save next to THIS script, so it works from any folder.
OUTPUT_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "assets", "sounds")


def write_wav(filename, samples):
    """Save a list of samples (floats from -1.0 to 1.0) as a 16-bit mono .wav file."""
    os.makedirs(OUTPUT_DIR, exist_ok=True)
    path = os.path.join(OUTPUT_DIR, filename)

    # Convert each float to a 16-bit integer and pack it as bytes.
    frames = b"".join(
        struct.pack("<h", int(max(-1.0, min(1.0, s)) * 32767)) for s in samples
    )

    with wave.open(path, "wb") as wav_file:
        wav_file.setnchannels(1)          # mono
        wav_file.setsampwidth(2)          # 16-bit
        wav_file.setframerate(SAMPLE_RATE)
        wav_file.writeframes(frames)
    print("Created", path)


def make_fire_sound():
    """Short 'pew': a square-ish tone sliding from high to low pitch."""
    duration = 0.15
    total = int(SAMPLE_RATE * duration)
    samples = []
    phase = 0.0
    for i in range(total):
        progress = i / total                      # 0.0 -> 1.0
        freq = 900 - 600 * progress               # slides 900 Hz -> 300 Hz
        phase += 2 * math.pi * freq / SAMPLE_RATE
        wave_value = 1.0 if math.sin(phase) >= 0 else -1.0   # square wave
        envelope = 1.0 - progress                 # fades out
        samples.append(wave_value * envelope * 0.3)
    return samples


def make_explosion_sound():
    """Short 'boom': noise that fades out quickly."""
    duration = 0.35
    total = int(SAMPLE_RATE * duration)
    samples = []
    seed = 12345        # simple pseudo-random generator (no extra imports)
    previous = 0.0
    for i in range(total):
        seed = (seed * 1103515245 + 12345) % 2147483648
        noise = (seed / 2147483648) * 2 - 1       # random value -1..1
        # Average with the previous sample to soften it (deeper rumble).
        smooth = (noise + previous) / 2
        previous = smooth
        progress = i / total
        envelope = (1.0 - progress) ** 2          # fast fade out
        samples.append(smooth * envelope * 0.9)
```

```
    return samples


def make_game_over_sound():
    """Sad descending notes."""
    notes = [(440, 0.20), (370, 0.20), (311, 0.20), (220, 0.55)]  # (Hz, seconds)
    samples = []
    for freq, length in notes:
        total = int(SAMPLE_RATE * length)
        for i in range(total):
            progress = i / total
            # Fade out over the last 40% of each note so notes don't click.
            envelope = 1.0 if progress < 0.6 else (1.0 - progress) / 0.4
            t = i / SAMPLE_RATE
            samples.append(math.sin(2 * math.pi * freq * t) * envelope * 0.4)
    return samples


if __name__ == "__main__":
    write_wav("fire.wav", make_fire_sound())
    write_wav("explosion.wav", make_explosion_sound())
    write_wav("game_over.wav", make_game_over_sound())
    print("Done! Sounds are in:", OUTPUT_DIR)
```

game/game_engine.py

```
import os
import pygame
import random
from .player import Player
from .enemy import EnemyGrid
from .bullet import Bullet

# Game Engine

WHITE = (255, 255, 255)
GREEN = (0, 200, 0)
RED = (220, 60, 60)

# ---------------------------------------------------------------
# SETTINGS - change these to tune the game
# ---------------------------------------------------------------
# Must match the FPS used in main.py, because the fire chance is
# calculated per frame.
FPS = 60

# After game over, key presses are ignored for this long so a player who
# is still mashing keys doesn't skip the screen by accident.
# Set to 0 to accept keys immediately.
GAME_OVER_INPUT_DELAY_SECONDS = 0.75

# Difficulty settings - all in ONE place, easy to tweak.
#   enemy_speed              : pixels the enemy grid moves per frame
#   enemy_bullets_per_second : average bullets the WHOLE grid fires per second
# The order here is the order shown in the menu (keys 1, 2, 3).
DIFFICULTY_SETTINGS = {
    "Easy":   {"enemy_speed": 1.0, "enemy_bullets_per_second": 0.5},
    "Medium": {"enemy_speed": 1.5, "enemy_bullets_per_second": 1.0},
    "Hard":   {"enemy_speed": 2.5, "enemy_bullets_per_second": 2.0},
}
DEFAULT_DIFFICULTY = "Medium"

# Sound files (created by generate_sounds.py).
# The folder is found relative to THIS file, so it works no matter
# which folder you start the game from.
PROJECT_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
SOUND_DIR = os.path.join(PROJECT_DIR, "assets", "sounds")
SOUND_FILES = {
    "fire": "fire.wav",
    "explosion": "explosion.wav",
    "game_over": "game_over.wav",
}
SOUND_VOLUME = 0.5  # 0.0 (silent) to 1.0 (full)
```

```
# Game states
STATE_PLAYING = "playing"
STATE_GAME_OVER = "game_over"
STATE_MENU = "menu"

# Choices on the Game Over screen
GAME_OVER_OPTIONS = ["Play again", "Exit"]


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        # Fonts are created once: normal, big (titles), small (hints)
        self.font = pygame.font.SysFont("Arial", 30)
        self.title_font = pygame.font.SysFont("Arial", 64, bold=True)
        self.small_font = pygame.font.SysFont("Arial", 24)

        # Names of the difficulties in menu order: ["Easy", "Medium", "Hard"]
        self.difficulty_names = list(DIFFICULTY_SETTINGS.keys())

        self.quit_requested = False  # main.py checks this to close the game

        # Load sounds once (they stay loaded for every replay).
        self.sounds = self._load_sounds()

        # Set up everything for the first game.
        self.start_new_game(DEFAULT_DIFFICULTY)

    # ------------------------------------------------------------
    # Sound
    # ------------------------------------------------------------
    def _load_sounds(self):
        """Load all sound effects. Returns a dict like {"fire": Sound or None}.
        If audio is not available, the game simply runs without sound."""
        sounds = {name: None for name in SOUND_FILES}

        # pygame.init() does not raise an error if there is no audio device,
        # so we make sure the mixer is really running here.
        try:
            if pygame.mixer.get_init() is None:
                pygame.mixer.init()
        except Exception as error:
            print("Audio not available, running without sound:", error)
            return sounds

        # Load each file separately, so one missing file doesn't
        # stop the others from loading.
        for name, filename in SOUND_FILES.items():
            path = os.path.join(SOUND_DIR, filename)
            try:
                sound = pygame.mixer.Sound(path)
                sound.set_volume(SOUND_VOLUME)
                sounds[name] = sound
            except Exception as error:
                print(f"Could not load sound '{filename}' "
                      f"(run generate_sounds.py first?): {error}")
        return sounds

    def _play(self, name):
        """Play a sound by name. Does nothing if it didn't load."""
        sound = self.sounds.get(name)
        if sound is not None:
            try:
                sound.play()
            except Exception:
                pass  # a sound problem must never crash the game

    # ------------------------------------------------------------
    # New game / state changes
    # ------------------------------------------------------------
    def start_new_game(self, difficulty):
        """Reset EVERYTHING and start a fresh game on the given difficulty."""
```

```
        settings = DIFFICULTY_SETTINGS[difficulty]
        self.difficulty = difficulty

        # Fresh player and enemies (enemy speed comes from the difficulty)
        self.player = Player(self.width // 2 - 20, self.height - 50, 40, 20)
        self.enemy_grid = EnemyGrid(self.width, speed=settings["enemy_speed"])

        # Empty the bullets and reset the shooting cooldown
        self.player_bullets = []
        self.enemy_bullets = []
        self._shoot_cooldown = 0

        # Chance (per frame) that the enemy grid fires ONE bullet.
        # Example: 1.0 bullet/sec / 60 frames/sec = about 1.7% per frame.
        self.enemy_fire_chance = settings["enemy_bullets_per_second"] / FPS

        # Score and state
        self.score = 0
        self.state = STATE_PLAYING

        # Game Over screen state
        self._game_over_timer = 0   # frames since the game ended
        self._game_over_delay = int(GAME_OVER_INPUT_DELAY_SECONDS * FPS)
        self._game_over_selected = 0   # 0 = Play again, 1 = Exit

        # Menu state (highlight starts on the default difficulty)
        self._menu_selected = self.difficulty_names.index(DEFAULT_DIFFICULTY)

    def _end_game(self):
        """Switch to the Game Over screen (and play the game over sound ONCE)."""
        # Guard: if the game is already over (e.g. a bullet hit AND the
        # enemies reached the bottom in the same frame), do nothing, so
        # the sound is never played twice.
        if self.state == STATE_GAME_OVER:
            return

        self.state = STATE_GAME_OVER
        self._game_over_timer = 0
        self._game_over_selected = 0
        self._play("game_over")

    # ------------------------------------------------------------
    # Input
    # ------------------------------------------------------------
    def handle_event(self, event):
        # Only key presses matter here (window close is handled in main.py)
        if event.type != pygame.KEYDOWN:
            return

        if self.state == STATE_MENU:
            self._handle_menu_key(event.key)
        elif self.state == STATE_GAME_OVER:
            self._handle_game_over_key(event.key)
        else:
            # Playing: Space shoots
            if event.key == pygame.K_SPACE and self._shoot_cooldown <= 0:
                bullet_x = self.player.center_x() - 2
                self.player_bullets.append(Bullet(bullet_x, self.player.y, direction=-1))
                self._shoot_cooldown = 15
                self._play("fire")

    def _handle_game_over_key(self, key):
        # Esc or Q always quits (even during the short input delay).
        if key in (pygame.K_ESCAPE, pygame.K_q):
            self.quit_requested = True
            return

        # Ignore other keys until the delay has passed.
        if self._game_over_timer < self._game_over_delay:
            return

        if key == pygame.K_UP:
            self._game_over_selected = (self._game_over_selected - 1) % len(GAME_OVER_OPTIONS)
        elif key == pygame.K_DOWN:
```

```
            self._game_over_selected = (self._game_over_selected + 1) % len(GAME_OVER_OPTIONS)
        elif key in (pygame.K_RETURN, pygame.K_KP_ENTER):
            if self._game_over_selected == 0:
                self.state = STATE_MENU          # Play again -> difficulty menu
            else:
                self.quit_requested = True       # Exit

    def _handle_menu_key(self, key):
        # Esc or Q quits from the menu too.
        if key in (pygame.K_ESCAPE, pygame.K_q):
            self.quit_requested = True
            return

        # Keys 1, 2, 3 pick a difficulty straight away.
        # (pygame.K_1, K_2, K_3 are consecutive numbers.)
        number_index = key - pygame.K_1
        if 0 <= number_index < len(self.difficulty_names):
            self.start_new_game(self.difficulty_names[number_index])
            return

        # Up/Down moves the highlight, Enter confirms.
        if key == pygame.K_UP:
            self._menu_selected = (self._menu_selected - 1) % len(self.difficulty_names)
        elif key == pygame.K_DOWN:
            self._menu_selected = (self._menu_selected + 1) % len(self.difficulty_names)
        elif key in (pygame.K_RETURN, pygame.K_KP_ENTER):
            self.start_new_game(self.difficulty_names[self._menu_selected])

    def handle_input(self):
        if self.state != STATE_PLAYING:
            return  # the player can only move while playing

        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT] or keys[pygame.K_a]:
            self.player.move(-self.player.speed, self.width)
        if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
            self.player.move(self.player.speed, self.width)

    # ------------------------------------------------------------
    # Game logic
    # ------------------------------------------------------------
    def update(self):
        if self.state == STATE_GAME_OVER:
            # The game logic is stopped, but we keep counting frames so
            # the input delay on the Game Over screen can finish.
            self._game_over_timer += 1
            return
        if self.state != STATE_PLAYING:
            return  # nothing to update in the menu

        if self._shoot_cooldown > 0:
            self._shoot_cooldown -= 1

        self.enemy_grid.move()

        # Enemy fire: roll ONCE per frame for the whole grid (not once per
        # enemy). If the roll succeeds, one random living enemy shoots.
        # This keeps the fire rate steady no matter how many enemies are left.
        alive = self.enemy_grid.alive_enemies()
        if alive and random.random() < self.enemy_fire_chance:
            shooter = random.choice(alive)
            bullet_x = shooter.x + shooter.width // 2
            self.enemy_bullets.append(Bullet(bullet_x, shooter.y + shooter.height, direction=1))

        # Move every bullet (each one remembers where it was last frame).
        for bullet in self.player_bullets:
            bullet.move()
        for bullet in self.enemy_bullets:
            bullet.move()

        # --- Player bullets vs enemies ---
        # We build a NEW list of bullets that survive instead of removing
        # items from the list we are looping over (that caused skipped checks).
        surviving_bullets = []
```

```
    for bullet in self.player_bullets:
        path = bullet.swept_rect()  # everything the bullet crossed this frame
        hit_enemy = None

        # alive_enemies() is called fresh for every bullet, so an enemy
        # killed by an earlier bullet this frame can't be hit again.
        for enemy in self.enemy_grid.alive_enemies():
            if path.colliderect(enemy.rect()):
                # If the path touches more than one enemy, take the
                # lowest one (largest y) - the bullet reached it first.
                if hit_enemy is None or enemy.y > hit_enemy.y:
                    hit_enemy = enemy

        if hit_enemy is not None:
            hit_enemy.alive = False   # at most ONE enemy per bullet
            self.score += 1           # the bullet is used up (not kept)
            self._play("explosion")   # enemy destroyed sound
        else:
            surviving_bullets.append(bullet)
    self.player_bullets = surviving_bullets

    # --- GAME OVER condition 1: an enemy bullet hits the player ---
    # Same swept check, so a fast bullet can't skip over the player.
    for bullet in self.enemy_bullets:
        if bullet.swept_rect().colliderect(self.player.rect()):
            self._end_game()
            break

    # Remove off-screen bullets AFTER collisions, so a bullet that hit
    # something and left the screen on the same frame still counts.
    self.player_bullets = [b for b in self.player_bullets if not b.off_screen(self.height)]
    self.enemy_bullets = [b for b in self.enemy_bullets if not b.off_screen(self.height)]

    # --- GAME OVER condition 2: the enemies reach the player's row ---
    if self.enemy_grid.reached_bottom(self.player.y):
        self._end_game()

# ------------------------------------------------------------
# Drawing
# ------------------------------------------------------------
def render(self, screen):
    # The difficulty menu has its own screen (no game behind it).
    if self.state == STATE_MENU:
        self._draw_menu(screen)
        return

    # Draw the game as usual (when the game is over it stays frozen
    # on screen behind the Game Over overlay).
    pygame.draw.rect(screen, GREEN, self.player.rect())

    for enemy in self.enemy_grid.alive_enemies():
        pygame.draw.rect(screen, WHITE, enemy.rect())

    for bullet in self.player_bullets:
        pygame.draw.rect(screen, WHITE, bullet.rect())
    for bullet in self.enemy_bullets:
        pygame.draw.rect(screen, RED, bullet.rect())

    score_text = self.font.render(f"Score: {self.score}", True, WHITE)
    screen.blit(score_text, (10, 10))

    # Current difficulty, top right
    level_text = self.small_font.render(self.difficulty, True, WHITE)
    screen.blit(level_text, level_text.get_rect(topright=(self.width - 10, 14)))

    if self.state == STATE_GAME_OVER:
        self._draw_game_over(screen)

def _draw_options(self, screen, labels, selected, first_y, gap):
    """Draw a centered list of options; the selected one is green with a box."""
    center_x = self.width // 2
    for i, label in enumerate(labels):
        is_selected = (i == selected)
        color = GREEN if is_selected else WHITE
```

```
            text = self.font.render(label, True, color)
            rect = text.get_rect(center=(center_x, first_y + i * gap))
            screen.blit(text, rect)
            if is_selected:
                pygame.draw.rect(screen, GREEN, rect.inflate(40, 16), 2)

    def _draw_game_over(self, screen):
        # Semi-transparent dark layer so the text is easy to read.
        overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
        overlay.fill((0, 0, 0, 180))  # last number = transparency (0-255)
        screen.blit(overlay, (0, 0))

        center_x = self.width // 2
        center_y = self.height // 2

        # "GAME OVER" title
        title = self.title_font.render("GAME OVER", True, RED)
        screen.blit(title, title.get_rect(center=(center_x, center_y - 100)))

        # Final score
        final_score = self.font.render(f"Final Score: {self.score}", True, WHITE)
        screen.blit(final_score, final_score.get_rect(center=(center_x, center_y - 30)))

        # Options - only shown once key presses are being accepted
        if self._game_over_timer >= self._game_over_delay:
            self._draw_options(screen, GAME_OVER_OPTIONS, self._game_over_selected,
                               first_y=center_y + 40, gap=60)
            hint = self.small_font.render("Up/Down + Enter to choose  |  Esc / Q to exit", True, WHITE)
            screen.blit(hint, hint.get_rect(center=(center_x, center_y + 170)))

    def _draw_menu(self, screen):
        center_x = self.width // 2

        title = self.title_font.render("DIFFICULTY", True, GREEN)
        screen.blit(title, title.get_rect(center=(center_x, 180)))

        # "1. Easy", "2. Medium", "3. Hard"
        labels = [f"{i + 1}. {name}" for i, name in enumerate(self.difficulty_names)]
        self._draw_options(screen, labels, self._menu_selected, first_y=300, gap=70)

        hint = self.small_font.render("Press 1/2/3, or Up/Down + Enter  |  Esc / Q to exit", True, WHITE)
        screen.blit(hint, hint.get_rect(center=(center_x, 560)))
```

To test it:

1. Run `python generate_sounds.py` once.
2. Start the game. Space should make a short "pew", and a hit enemy should make a "boom".
3. When you die, the descending game-over tune plays once.

If you start the game before generating the files, it runs silently and prints a warning for each missing file.

User

export the entire chat in pdf form